



SCHOOL OF COMPUTING

Department of Artificial Intelligence and Machine Learning

SOFTWARE MANAGEMENT SYSTEM

A Cornerstone Project Report

BATCH NUMBER : 9

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

M. SRUJANA - AIML (25B11AI651)

S. BHAVYA TULASI - AIML (26B61AIA86)

T. TEJASRI - AIML (25B11AI910)

T. SANJANA - AIML (25B11AIB85)

Under the guidance of

Dr. P. Kishorekumar, PhD.

Associate Professor, Department of AIML

ADITYA UNIVERSITY, SURAMPALEM

Academic Year: 2026 - 2027

SCHOOL OF COMPUTING

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the Cornerstone project work entitled "***HOSPITAL MANAGEMENT DATABASE SYSTEM***" is submitted by

M SRUJANA - AIML

(25B11AI651)

S BHAVYA TULASI - AIML

(26B61AIA86)

T TEJA SRI - AIML

(25B11AI910)

T SANJANA - AIML

(25B11AIB85)

In partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning for the academic year 2026-2027.

Project Guide

Dr. P. Kishorekumar, PhD.

Associate Professor, Department of AIML

Head of the Department:

Dr. Kovvuri N Bhargavi

Associate Professor & HOD

DECLARATION

We hereby declare that the Cornerstone project entitled "**HOSPITAL MANAGEMENT DATABASE SYSTEM**" is original and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfillment of the requirements for the award of the B.Tech. Degree in Artificial Intelligence and Machine Learning. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date:

By

M SRUJANA - AIML

(25B11AI651)

S BHAVYA TULASI - AIML

(26B61AIA86)

T TEJA SRI - AIML

(25B11AI910)

T SANJANA - AIML

(25B11AIB85)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Dr. P. Kishorekumar, PhD. Associate Professor, Department of AIML**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. B. Manikyala Rao, Associate Professor, Dr. R. Suneetha, Assistant Professor and Dr. D. Veeraswamy, Assistant Professor** for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. K. Naga Bhargavi**, Associate Professor & HOD, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

BY

M SRUJANA - AIML (25B11AI651)

S BHAVYA TULASI - AIML (26B61AIA86)

T TEJA SRI - AIML (25B11AI910)

T SANJANA - AIML (25B11AIB85)

CERTIFICATE

This is to certify that the project entitled “Software Management System” is a bonafide DBMS project carried out as part of the academic requirements. The project demonstrates the analysis, design, normalization and SQL implementation of a relational database for managing software, users, departments, licenses, installations and maintenance records.

Role	Name / Signature
Project Guide	_____
Head of the Department	_____
Project Team	_____
Date	_____

DECLARATION

We hereby declare that the project documentation entitled “Software Management System” has been prepared as part of our academic DBMS project work. The documentation presents the database design, entity relationships, relational schema, normalization, SQL implementation and query operations of the proposed system.

We further declare that the project work has been prepared for academic purposes and that the database design is intended to demonstrate the practical application of Database Management System concepts.

Student / Team Member	Signature
1. _____	_____
2. _____	_____
3. _____	_____
4. _____	_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, faculty members and department for providing the opportunity and guidance to develop the Software Management System DBMS project.

We are thankful for the technical support and academic suggestions that helped us understand database design, ER modeling, relational schemas, normalization and SQL implementation. We also thank our team members for their cooperation during the design, implementation and documentation stages of the project.

Finally, we thank everyone who supported us directly or indirectly in completing this project documentation.

ABSTRACT

The Software Management System is designed to organize and manage information related to software used within an organization. In a practical environment, an organization may have many departments, users, software products, licenses, installations and maintenance activities. Maintaining such information manually or through disconnected files can result in duplication, inconsistent records, difficulty in tracking licenses and problems in identifying installed software.

The proposed system uses a centralized relational database to store and connect software-management information. The database contains six major entities — Department, Users, Software, License, Installation and Maintenance. Departments are associated with users and software, while software records are connected with licenses, installations and maintenance activities. Installation records also connect users with the software they use.

The database is designed with Primary Keys and Foreign Keys to maintain referential integrity. The design is normalized up to Third Normal Form (3NF) to reduce redundancy and update anomalies. The SQL implementation covers DDL for database and table creation, DML for sample data insertion and modification, and DQL for retrieval, joins, aggregation, subqueries and view-based reporting.

The resulting database provides a structured foundation for software inventory management, user-wise installation tracking, license monitoring and maintenance record management.

LIST OF FIGURES

- Figure 1: ER Diagram of the Software Management System
- Figure 2: Manual / Conceptual ER Representation
- Figure 3: MySQL Database and Table Structure Screenshot
- Figure 4: Sample SQL Query Output Screenshot

LIST OF TABLES

- Table 1: Major Database Entities
- Table 2: Entity Relationships and Cardinality
- Table 3: Relational Schema Summary
- Table 4: Department Table Structure
- Table 5: Users Table Structure
- Table 6: Software Table Structure
- Table 7: License Table Structure
- Table 8: Installation Table Structure
- Table 9: Maintenance Table Structure
- Table 10: Keys and Constraints Summary

ACRONYMS

Acronym	Meaning
DBMS	Database Management System
ER	Entity Relationship
SQL	Structured Query Language
DDL	Data Definition Language
DML	Data Manipulation Language
DQL	Data Query Language
DCL	Data Control Language
TCL	Transaction Control Language
PK	Primary Key
FK	Foreign Key
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form

1. INTRODUCTION

1.1 Brief Information about the Project

Organizations depend on software applications for communication, productivity, development, documentation, data analysis and many other activities. As the number of software products and users increases, it becomes important to maintain structured records of the software available in the organization.

The Software Management System models the major information required to manage software resources. The project focuses on identifying entities, attributes, Primary Keys, Foreign Keys and relationships, converting the conceptual model into a relational schema, and implementing the database using MySQL.

1.2 Motivation and Contribution of the Project

The project is motivated by the need to maintain software information in an organized and centralized manner. Separate files can make it difficult to know which software is installed for a particular user, which department manages the software, whether a license is still valid, and when maintenance was last performed.

- Centralizing software-related records in one relational database.
- Maintaining department and user information in an organized structure.
- Tracking software licenses, expiry dates and quantities.
- Tracking which software is installed for each user.
- Maintaining software maintenance history.
- Reducing duplicate and inconsistent information using normalization.
- Supporting retrieval and reporting through SQL queries.

1.3 Objectives of the Project

- Identify the major entities involved in software management.
- Identify suitable attributes for each entity.
- Establish relationships between the identified entities.
- Design an Entity–Relationship (ER) diagram.
- Convert the ER model into a relational database schema.
- Identify suitable Primary Keys and Foreign Keys.
- Track software licenses and expiry information.
- Track software installation details for users.
- Maintain software maintenance records.
- Reduce data duplication and improve data consistency.

1.4 Scope of the Project

The scope of the project covers database-oriented management of software resources. The system maintains department information, user details, software information, license records, installation records and maintenance activities. It can serve as the database backend for an organization's software administration application and can later be extended with modules such as asset management, purchase records, vendor management, audit logs and automated notifications.

2.SYSTEM ANALYSIS

2. 2.1 Existing System

- Software records may be maintained manually or in separate spreadsheets.
- Information about users and departments may be stored independently.
- Software installation details can be difficult to track centrally.
- License expiry dates may be missed when there is no centralized database.
- Maintenance history may not be consistently recorded.
- Searching for software usage information can take more time.
- Duplicate and inconsistent records can occur when information is maintained separately.

2.2 Proposed System

The proposed system uses a centralized relational database that connects software-management records through well-defined entities, relationships, Primary Keys and Foreign Keys. It organizes information into six related tables and uses constraints such as NOT NULL, UNIQUE, CHECK and Foreign Key rules to maintain valid data.

- Centralized software-management database with six major entities.
- Department and user management.
- Software inventory management.
- License and expiry tracking.
- User-wise software installation tracking.
- Software maintenance record management.
- SQL-based retrieval and reporting.
- Referential integrity across related tables.

2.3 Functional Requirements

Module	Functional Requirement
Department Management	Store and maintain department name, location and manager information.
User Management	Store user name, email, role and department association.
Software Management	Store software name, version, vendor and category.
License Management	Maintain license type, expiry date and quantity for software.
Installation Management	Record software installations for users, installation date and status.
Maintenance Management	Record maintenance date, type and remarks for software.
Search and Reporting	Retrieve software, user, license, installation and maintenance information using SQL queries.

2.4 Non-Functional Requirements

- Performance – database queries should retrieve software and user information efficiently.
- Integrity – invalid Foreign Key references and duplicate key values must be prevented.
- Security – access to software-management information should be limited to authorized users.
- Usability – the table structure should be understandable for administrators.
- Reliability – database operations should preserve a consistent database state.
- Scalability – the design should support growth in departments, users and software records.

2.5 Requirements Specification

Category	Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or equivalent supported platform.
Database	MySQL Relational Database Management System.
Development Tool	MySQL Workbench / MySQL client.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
DEPARTMENT	Stores department name, location and manager information.
USERS	Stores user details, role and department reference.
SOFTWARE	Stores software name, version, vendor and category.
LICENSE	Stores license information for software, including type, expiry date and quantity.
INSTALLATION	Stores the relationship between users and software with installation date and status.
MAINTENANCE	Stores software maintenance date, type and remarks.

3.2 ER Model

The ER model represents the six major entities and their relationships. DEPARTMENT is associated with USERS and SOFTWARE. SOFTWARE is associated with LICENSE, INSTALLATION and MAINTENANCE. INSTALLATION connects a USER with SOFTWARE, allowing the system to record which software is installed for a particular user.

Figure 1: ER Diagram of the Software Management System

3.3 Relationships and Cardinality

Relationship	Cardinality	Meaning
DEPARTMENT – USERS	1 : M	One department can have many users.
DEPARTMENT – SOFTWARE	1 : M	One department can be associated with many software records.
SOFTWARE – LICENSE	1 : M	One software can have many license records.
SOFTWARE – INSTALLATION	1 : M	One software can have many installation records.
USERS – INSTALLATION	1 : M	One user can have many software installations.
SOFTWARE – MAINTENANCE	1 : M	One software can have many maintenance records.

4. ER DIAGRAM

The ER diagram shows the six entities of the Software Management System and their one-to-many relationships. Primary Keys and Foreign Keys are identified inside the entity structures. The diagram provides the conceptual foundation for converting the project into relational tables.

ER DIAGRAM – SOFTWARE MANAGEMENT SYSTEM

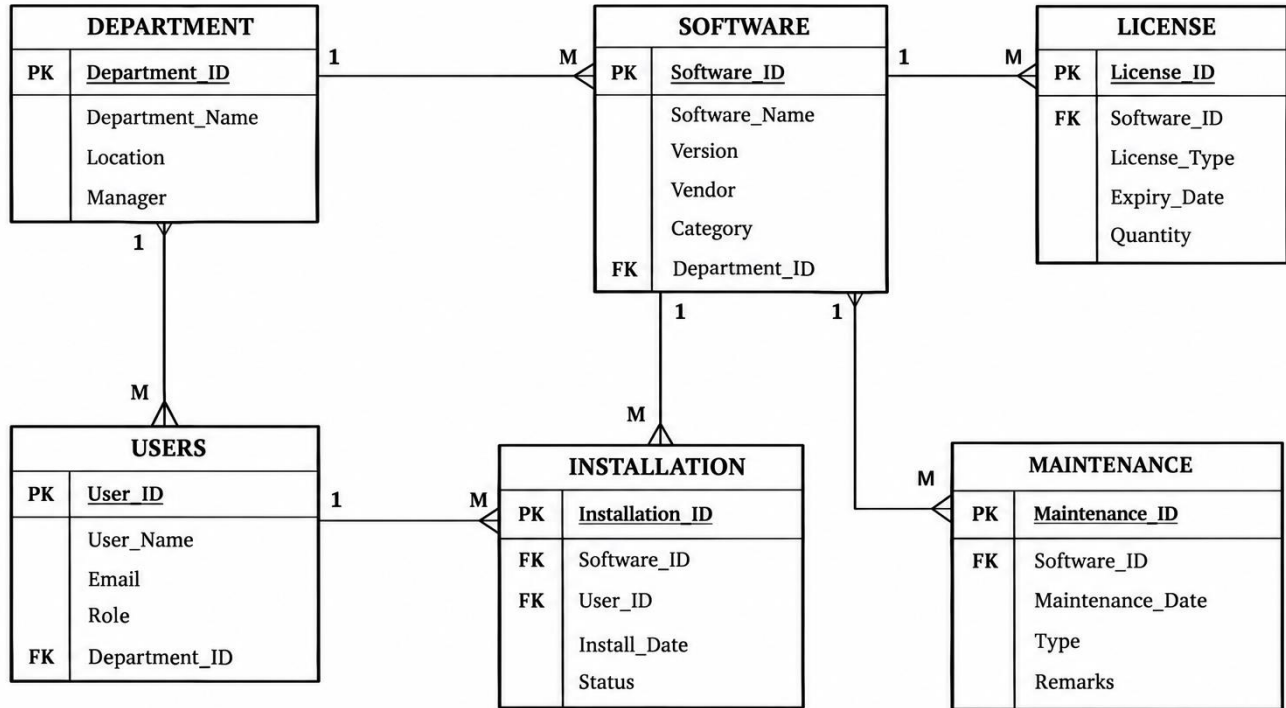
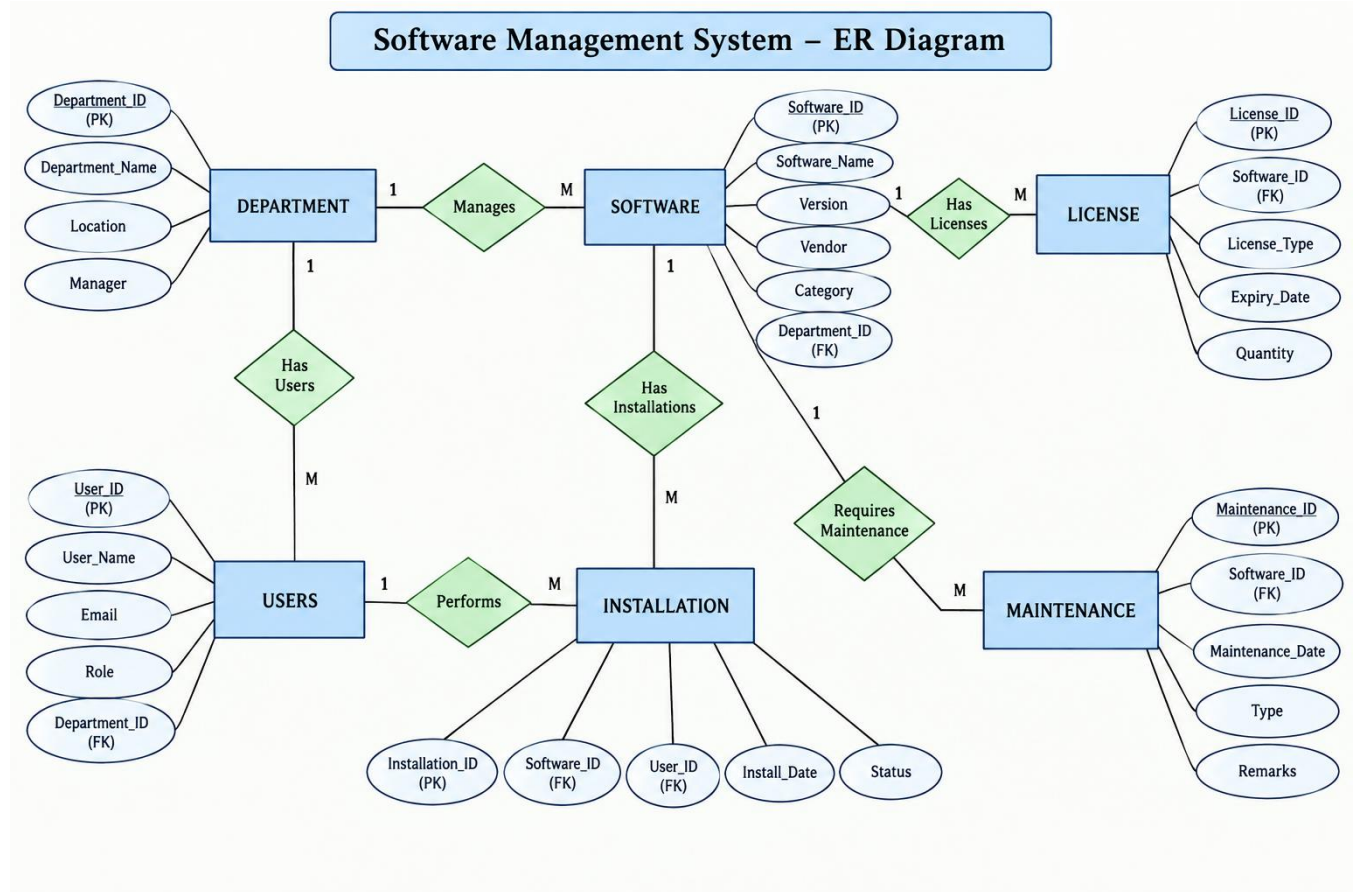


Figure 2: ER Diagram of the Software Management System

For the final project submission, a screenshot of the ER diagram created in MySQL Workbench can be placed in this section if required by the project review format.



5. RELATIONAL SCHEMA

The relational schema follows the dependency order so that parent tables are created before tables containing their Foreign Keys.

Relation	Relational Schema	Keys
DEPARTMENT	Department(Department_ID, Department_Name, Location, Manager)	PK: Department_ID
USERS	Users(User_ID, User_Name, Email, Role, Department_ID)	PK: User_ID; FK: Department_ID
SOFTWARE	Software(Software_ID, Software_Name, Version, Vendor, Category, Department_ID)	PK: Software_ID; FK: Department_ID
LICENSE	License(License_ID, Software_ID, License_Type, Expiry_Date, Quantity)	PK: License_ID; FK: Software_ID
INSTALLATION	Installation(Installation_ID, Software_ID, User_ID, Install_Date, Status)	PK: Installation_ID; FK: Software_ID, User_ID
MAINTENANCE	Maintenance(Maintenance_ID, Software_ID, Maintenance_Date, Type, Remarks)	PK: Maintenance_ID; FK: Software_ID

Department is referenced by Users and Software. Software is referenced by License, Installation and Maintenance. Installation references both Software and Users, representing software usage at the user level.

6. TABLE DESIGN

6.1 Department Table

Attribute	Data Type	Constraint	Description
Department_ID	INT	PK, AUTO_INCREMENT	Unique department identifier
Department_Name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the department
Location	VARCHAR(100)	NOT NULL	Department location
Manager	VARCHAR(100)	NOT NULL	Department manager

6.2 Users Table

Attribute	Data Type	Constraint	Description
User_ID	INT	PK, AUTO_INCREMENT	Unique user identifier
User_Name	VARCHAR(100)	NOT NULL	User's name
Email	VARCHAR(100)	NOT NULL, UNIQUE	User email address
Role	VARCHAR(50)	NOT NULL	Role of the user
Department_ID	INT	NOT NULL, FK	References Department

6.3 Software Table

Attribute	Data Type	Constraint	Description
Software_ID	INT	PK, AUTO_INCREMENT	Unique software identifier
Software_Name	VARCHAR(100)	NOT NULL	Software name
Version	VARCHAR(50)	NOT NULL	Software version
Vendor	VARCHAR(100)	NOT NULL	Software vendor
Category	VARCHAR(50)	NOT NULL	Software category
Department_ID	INT	NOT NULL, FK	References Department

6.4 License Table

Attribute	Data Type	Constraint	Description
License_ID	INT	PK, AUTO_INCREMENT	Unique license identifier
Software_ID	INT	NOT NULL, FK	References Software
License_Type	VARCHAR(50)	NOT NULL	Type of license
Expiry_Date	DATE	NOT NULL	License expiry date

Quantity	INT	NOT NULL, CHECK (>=0)	Number of licensed copies
----------	-----	--------------------------	------------------------------

6.5 Installation Table

Attribute	Data Type	Constraint	Description
Installation_ID	INT	PK, AUTO_INCREMENT	Unique installation identifier
Software_ID	INT	NOT NULL, FK	References Software
User_ID	INT	NOT NULL, FK	References Users
Install_Date	DATE	NOT NULL	Date of installation
Status	VARCHAR(30)	NOT NULL	Installation status

6.6 Maintenance Table

Attribute	Data Type	Constraint	Description
Maintenance_ID	INT	PK, AUTO_INCREMENT	Unique maintenance identifier
Software_ID	INT	NOT NULL, FK	References Software
Maintenance_Date	DATE	NOT NULL	Date of maintenance
Type	VARCHAR(50)	NOT NULL	Type of maintenance
Remarks	VARCHAR(255)	NULL	Maintenance details or remarks

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Department_ID, User_ID, Software_ID, License_ID, Installation_ID, Maintenance_ID
Foreign Key	Users.Department_ID; Software.Department_ID; License.Software_ID; Installation.Software_ID; Installation.User_ID; Maintenance.Software_ID
NOT NULL	Required names, identifiers, roles, dates, status and references
UNIQUE	Department_Name and Users.Email
CHECK	License.Quantity should be zero or greater
DEFAULT	Installation.Status may use a default value such as 'Active'

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The Software Management System is designed up to Third Normal Form (3NF).

7.1 Unnormalized Form (UNF)

A single software-management record could contain department details, user details, software details, license details, installation information and maintenance details in one large record. If a software product has multiple installations or maintenance activities, repeating groups would be created. This can cause redundant information and inconsistent updates.

7.2 First Normal Form (1NF)

In 1NF, repeating groups are removed and every attribute contains atomic values. Each installation and maintenance activity is represented as a separate record rather than storing multiple values in one field.

7.3 Second Normal Form (2NF)

In 2NF, partial dependencies are removed. Department attributes depend on Department_ID, user attributes depend on User_ID, software attributes depend on Software_ID, and license, installation and maintenance attributes depend on their respective identifiers. This motivates the separation of the entities into independent relations.

7.4 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Department details are stored only in Department rather than being repeated in Users or Software. Software details are stored only in Software rather than being repeated in License, Installation or Maintenance. The resulting six relations correspond to the tables implemented in the SQL database.

Relation	Reason for Separation
Department	Stores department-specific attributes once.
Users	Stores user-specific attributes and a Department_ID reference.
Software	Stores software-specific attributes and its department reference.
License	Stores license-specific information linked to Software.
Installation	Stores user-software installation information.
Maintenance	Stores maintenance-specific information linked to Software.

8. SQL IMPLEMENTATION

The SQL implementation creates the software_management_db database and six related tables. Tables are created in dependency order: DEPARTMENT first, followed by USERS and SOFTWARE, and then LICENSE, INSTALLATION and MAINTENANCE.

8.1 DDL – Data Definition Language

```
CREATE DATABASE software_management_db;
USE software_management_db;
```

```
CREATE TABLE DEPARTMENT (
    Department_ID INT AUTO_INCREMENT PRIMARY KEY,
    Department_Name VARCHAR(100) NOT NULL UNIQUE,
    Location VARCHAR(100) NOT NULL,
    Manager VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE USERS (
    User_ID INT AUTO_INCREMENT PRIMARY KEY,
    User_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Role VARCHAR(50) NOT NULL,
    Department_ID INT NOT NULL,
    FOREIGN KEY (Department_ID)
        REFERENCES DEPARTMENT(Department_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
CREATE TABLE SOFTWARE (
    Software_ID INT AUTO_INCREMENT PRIMARY KEY,
    Software_Name VARCHAR(100) NOT NULL,
    Version VARCHAR(50) NOT NULL,
    Vendor VARCHAR(100) NOT NULL,
    Category VARCHAR(50) NOT NULL,
    Department_ID INT NOT NULL,
    FOREIGN KEY (Department_ID)
        REFERENCES DEPARTMENT(Department_ID)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
);
```

```
CREATE TABLE LICENSE (  
    License_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Software_ID INT NOT NULL,  
    License_Type VARCHAR(50) NOT NULL,  
    Expiry_Date DATE NOT NULL,  
    Quantity INT NOT NULL CHECK (Quantity >= 0),  
    FOREIGN KEY (Software_ID)  
        REFERENCES SOFTWARE(Software_ID)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
);
```

```
CREATE TABLE INSTALLATION (  
    Installation_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Software_ID INT NOT NULL,  
    User_ID INT NOT NULL,  
    Install_Date DATE NOT NULL,  
    Status VARCHAR(30) NOT NULL DEFAULT 'Active',  
    FOREIGN KEY (Software_ID)  
        REFERENCES SOFTWARE(Software_ID)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    FOREIGN KEY (User_ID)  
        REFERENCES USERS(User_ID)  
        ON UPDATE CASCADE ON DELETE CASCADE  
);
```

```
CREATE TABLE MAINTENANCE (  
    Maintenance_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Software_ID INT NOT NULL,  
    Maintenance_Date DATE NOT NULL,  
    Type VARCHAR(50) NOT NULL,  
    Remarks VARCHAR(255),  
    FOREIGN KEY (Software_ID)  
        REFERENCES SOFTWARE(Software_ID)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
);
```

8.2 DML – Sample Data

INSERT INTO DEPARTMENT

```
(Department_Name, Location, Manager) VALUES
('IT Department', 'Block A, Floor 1', 'Ravi Kumar'),
('CSE Department', 'Block A, Floor 2', 'Sita Devi'),
('AIML Department', 'Block B, Floor 1', 'Anil Reddy'),
('Administration', 'Block B, Floor 2', 'Lakshmi Priya');
```

INSERT INTO USERS

```
(User_Name, Email, Role, Department_ID) VALUES
('Bhavya', 'bhavya@example.com', 'Student', 3),
('Srujana', 'srujana@example.com', 'Faculty', 2),
('Pravallika', 'pravallika@example.com', 'Faculty', 3),
('Venkata Raju', 'raju@example.com', 'Admin', 1);
```

INSERT INTO SOFTWARE

```
(Software_Name, Version, Vendor, Category, Department_ID) VALUES
('Microsoft Office', '2024', 'Microsoft', 'Productivity', 1),
('MySQL', '8.0', 'Oracle', 'Database', 1),
('Visual Studio Code', '1.100', 'Microsoft', 'Development', 3),
('Python', '3.13', 'Python Software Foundation', 'Programming', 3);
```

INSERT INTO LICENSE

```
(Software_ID, License_Type, Expiry_Date, Quantity) VALUES
(1, 'Commercial', '2027-12-31', 100),
(2, 'Community', '2027-06-30', 50),
(3, 'Free', '2028-01-31', 100),
(4, 'Open Source', '2028-12-31', 100);
```

INSERT INTO INSTALLATION

```
(Software_ID, User_ID, Install_Date, Status) VALUES
(1, 1, '2026-08-01', 'Active'),
(2, 1, '2026-08-02', 'Active'),
(3, 1, '2026-08-03', 'Active'),
(4, 2, '2026-08-04', 'Active');
```

INSERT INTO MAINTENANCE

```
(Software_ID, Maintenance_Date, Type, Remarks) VALUES
(1, '2026-09-01', 'Update', 'Security update installed'),
(2, '2026-09-05', 'Backup', 'Database backup completed'),
(3, '2026-09-10', 'Update', 'Editor updated'),
(4, '2026-09-15', 'Check', 'Environment verified');
```

8.3 CRUD Operations

Operation	Purpose	Example
INSERT	Add a new user or software record.	INSERT INTO USERS ...
SELECT	Retrieve records from the database.	SELECT * FROM SOFTWARE;
UPDATE	Modify existing information.	UPDATE INSTALLATION SET Status='Inactive' ...
DELETE	Remove a record when permitted by constraints.	DELETE FROM MAINTENANCE WHERE Maintenance_ID=4;

9. SQL QUERIES & OUTPUT PLACEHOLDERS

The following query set demonstrates CRUD operations, joins, aggregation, subqueries and views. Actual MySQL Workbench screenshots can be inserted below each output placeholder during final project submission.

Q1. List all software

```
SELECT * FROM SOFTWARE;
```

Output: _____

Q2. Find software in the AIML department

```
SELECT S.Software_Name, S.Version, S.Vendor
FROM SOFTWARE S
JOIN DEPARTMENT D ON S.Department_ID = D.Department_ID
WHERE D.Department_Name = 'AIML Department';
```

Output: _____

Q3. List users with their department

```
SELECT U.User_Name, U.Email, U.Role, D.Department_Name
FROM USERS U
JOIN DEPARTMENT D ON U.Department_ID = D.Department_ID;
```

Output: _____

Q4. List software installed for each user

```
SELECT U.User_Name, S.Software_Name, I.Install_Date, I.Status
FROM INSTALLATION I
JOIN USERS U ON I.User_ID = U.User_ID
JOIN SOFTWARE S ON I.Software_ID = S.Software_ID;
```

Output: _____

Q5. List active installations

```
SELECT * FROM INSTALLATION WHERE Status = 'Active';
```

Output: _____

Q6. Find licenses expiring before a given date

```
SELECT S.Software_Name, L.License_Type, L.Expiry_Date, L.Quantity
FROM LICENSE L
JOIN SOFTWARE S ON L.Software_ID = S.Software_ID
WHERE L.Expiry_Date < '2028-01-01';
```

Output: _____

Q7. Count software by department

```
SELECT D.Department_Name, COUNT(S.Software_ID) AS Software_Count
FROM DEPARTMENT D
```

```
LEFT JOIN SOFTWARE S ON D.Department_ID = S.Department_ID
GROUP BY D.Department_ID, D.Department_Name;
```

Output: _____

Q8. Count installations for each software

```
SELECT S.Software_Name, COUNT(I.Installation_ID) AS Installation_Count
FROM SOFTWARE S
LEFT JOIN INSTALLATION I ON S.Software_ID = I.Software_ID
GROUP BY S.Software_ID, S.Software_Name;
```

Output: _____

Q9. Find software with more than one installation

```
SELECT S.Software_Name, COUNT(I.Installation_ID) AS Installation_Count
FROM SOFTWARE S
JOIN INSTALLATION I ON S.Software_ID = I.Software_ID
GROUP BY S.Software_ID, S.Software_Name
HAVING COUNT(I.Installation_ID) > 1;
```

Output: _____

Q10. Find software having the maximum license quantity

```
SELECT S.Software_ID, S.Software_Name
FROM SOFTWARE S
JOIN LICENSE L ON S.Software_ID = L.Software_ID
WHERE L.Quantity = (SELECT MAX(Quantity) FROM LICENSE);
```

Output: _____

Q11. Create a software-installation view

```
CREATE VIEW Software_Installation_View AS
SELECT U.User_Name, U.Role, S.Software_Name, S.Version,
       I.Install_Date, I.Status
FROM USERS U
JOIN INSTALLATION I ON U.User_ID = I.User_ID
JOIN SOFTWARE S ON I.Software_ID = S.Software_ID;
```

Output: _____

Q12. Query the view

```
SELECT * FROM Software_Installation_View;
```

Output: _____

10. SQL IMPLEMENTATION SCREENSHOTS

The reference documentation uses actual SQL output screenshots after the query sections. For this Software Management System documentation, the following spaces are provided for screenshots captured from MySQL Workbench after executing the SQL.

Figure 3.1: MySQL database created successfully

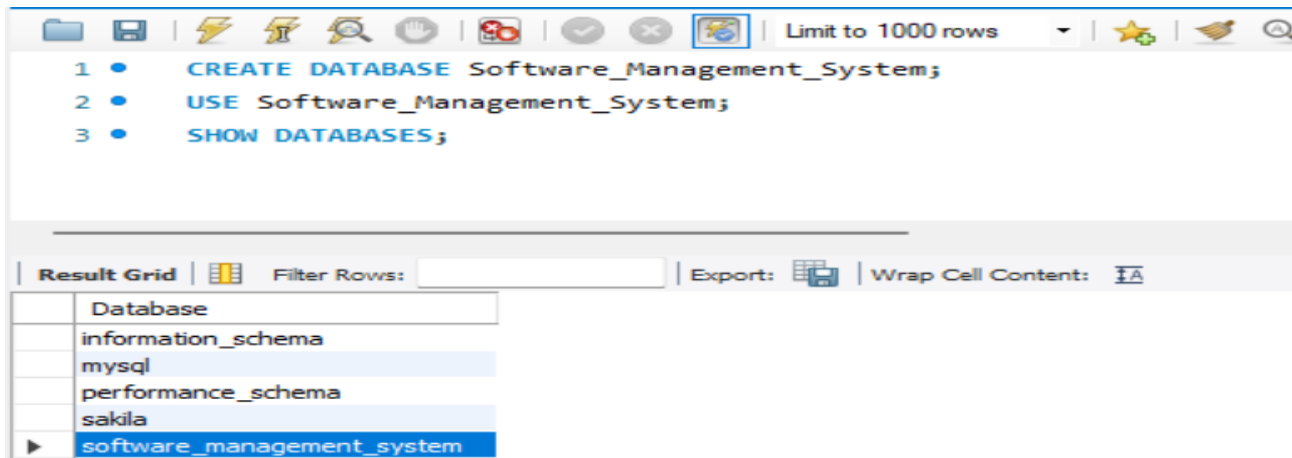


Figure 3.2: Tables created successfully

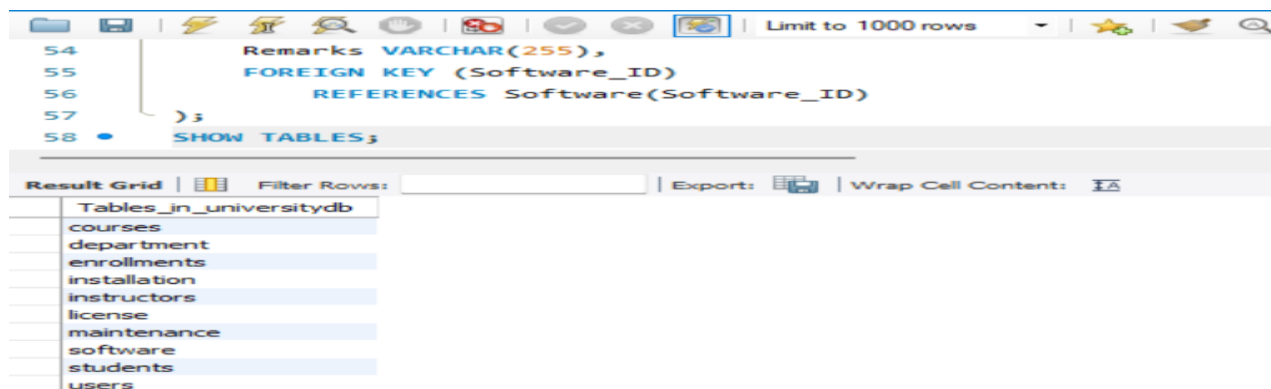


Figure 3.3: Sample data inserted into tables

107 • `SELECT * FROM Users;`
 108 • `SELECT * FROM Software;`
 109 • `SELECT * FROM License;`
 110 • `SELECT * FROM Installation;`
 111 • `SELECT * FROM Maintenance;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Maintenance_ID	Software_ID	Maintenance_Date	Type	Remarks
601	301	2026-03-10	Update	Latest security updates installed
602	303	2026-03-15	Backup	Database backup completed
603	304	2026-04-05	Repair	License configuration repaired
604	302	2026-04-10	Update	Python packages updated
605	305	2026-04-20	Update	VS Code updated successfully
NULL	NULL	NULL	NULL	NULL

Figure 3.4: SELECT query output

111 • `SELECT * FROM Maintenance;`
 112 • `SELECT`
 113 `Software_ID,`
 114 `Software_Name,`
 115 `Version,`
 116 `Vendor,`
 117 `Category`
 118 `FROM Software;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Software_ID	Software_Name	Version	Vendor	Category
301	Microsoft Office	2021	Microsoft	Productivity
302	Python	3.12	Python Software Foundation	Programming
303	MySQL	8.0	Oracle	Database
304	MATLAB	2024	MathWorks	Engineering
305	Visual Studio Code	1.95	Microsoft	Development

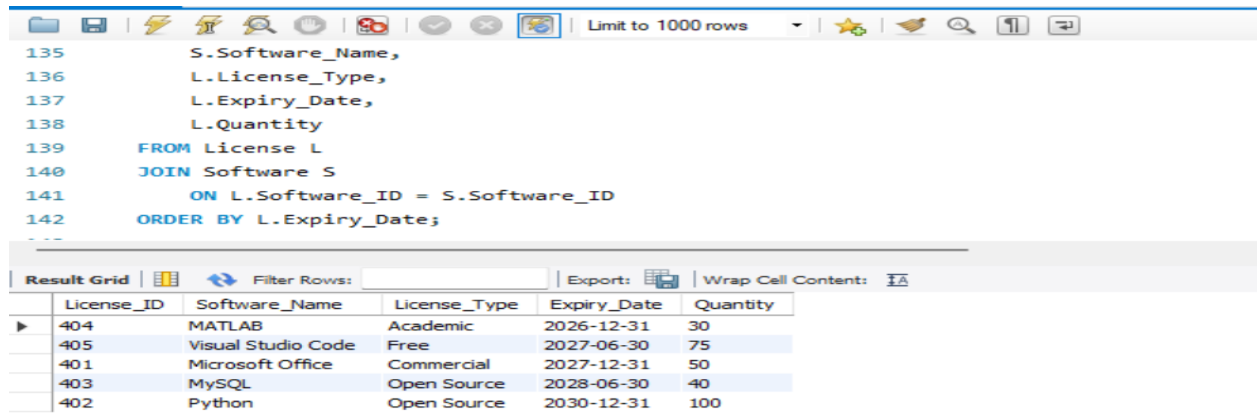
Figure 3.5: JOIN query output

125 `L.License_Type,`
 126 `L.Expiry_Date,`
 127 `L.Quantity`
 128 `FROM Software S`
 129 `JOIN Department D`
 130 `ON S.Department_ID = D.Department_ID`
 131 `JOIN License L`
 132 `ON S.Software_ID = L.Software_ID;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Software_ID	Software_Name	Version	Vendor	Department_Name	License_Type	Expiry_Date	Quantity
301	Microsoft Office	2021	Microsoft	Computer Science	Commercial	2027-12-31	50
303	MySQL	8.0	Oracle	Computer Science	Open Source	2028-06-30	40
302	Python	3.12	Python Software Foundation	Artificial Intelligence	Open Source	2030-12-31	100
305	Visual Studio Code	1.95	Microsoft	Artificial Intelligence	Free	2027-06-30	75
304	MATLAB	2024	MathWorks	Electronics	Academic	2026-12-31	30

Figure 3.6: License-expiry query output



The screenshot shows a database query editor with a SQL query and its result grid. The query is as follows:

```

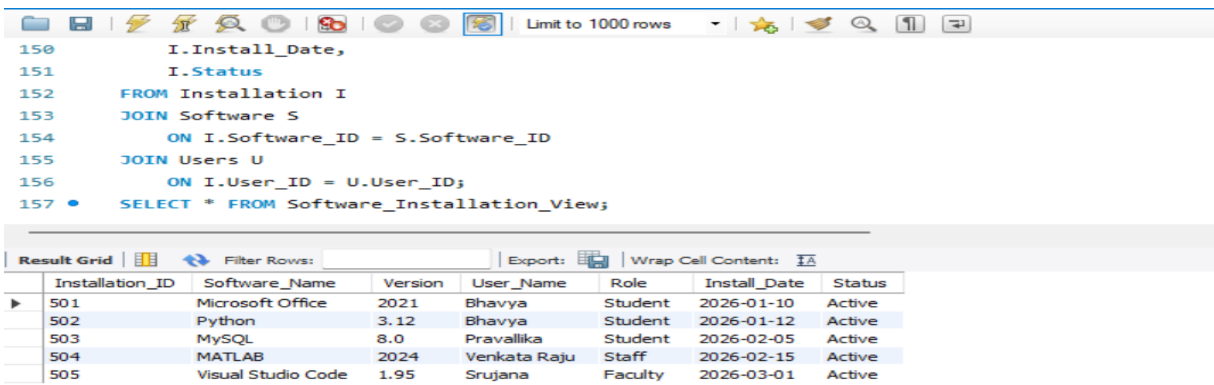
135     S.Software_Name,
136     L.License_Type,
137     L.Expiry_Date,
138     L.Quantity
139 FROM License L
140 JOIN Software S
141     ON L.Software_ID = S.Software_ID
142 ORDER BY L.Expiry_Date;

```

The result grid displays the following data:

License_ID	Software_Name	License_Type	Expiry_Date	Quantity
404	MATLAB	Academic	2026-12-31	30
405	Visual Studio Code	Free	2027-06-30	75
401	Microsoft Office	Commercial	2027-12-31	50
403	MySQL	Open Source	2028-06-30	40
402	Python	Open Source	2030-12-31	100

Figure 3.7: Software-installation view output



The screenshot shows a database query editor with a SQL query and its result grid. The query is as follows:

```

150     I.Install_Date,
151     I.Status
152 FROM Installation I
153 JOIN Software S
154     ON I.Software_ID = S.Software_ID
155 JOIN Users U
156     ON I.User_ID = U.User_ID;
157 SELECT * FROM Software_Installation_View;

```

The result grid displays the following data:

Installation_ID	Software_Name	Version	User_Name	Role	Install_Date	Status
501	Microsoft Office	2021	Bhavya	Student	2026-01-10	Active
502	Python	3.12	Bhavya	Student	2026-01-12	Active
503	MySQL	8.0	Pravallika	Student	2026-02-05	Active
504	MATLAB	2024	Venkata Raju	Staff	2026-02-15	Active
505	Visual Studio Code	1.95	Srujana	Faculty	2026-03-01	Active

11. CONCLUSION

The Software Management System provides a structured relational database solution for managing software-related information within an organization. The database organizes departments, users, software, licenses, installations and maintenance activities into six related tables.

The use of Primary Keys, Foreign Keys, UNIQUE, NOT NULL and CHECK constraints helps maintain data integrity. The normalization process up to 3NF reduces unnecessary duplication and update anomalies. The SQL implementation demonstrates database creation, table creation, sample data insertion, modification, deletion, retrieval, joins, aggregation, subqueries and views.

The project therefore provides a practical DBMS foundation for software inventory and administration. It can be used as the backend database for a future software management application.

11.1 Project Advantages

- Centralized management of software information.
- Easy tracking of users and their departments.
- Software license quantity and expiry tracking.
- User-wise software installation tracking.
- Maintenance history management.
- Reduced data redundancy through normalization.
- Fast information retrieval using SQL queries.
- Easy extension for additional software-management modules.

11.2 Future Enhancements

- Role-based access for administrators, faculty and authorized users.
- Automatic license-expiry notifications.
- Software purchase and vendor-management module.
- Hardware and software asset-management integration.
- Automated dashboards for software usage and license status.
- Audit logging for installation and maintenance changes.
- Backup and recovery mechanisms.
- Web-based interface connected to the MySQL database.

12. REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, Database System Concepts, McGraw Hill.
2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
3. MySQL Reference Manual, Oracle Corporation.
4. MySQL Workbench documentation and classroom DBMS materials.

Reference structure used for this documentation: the uploaded DBMS Cornerstone Project document, which organizes the report into cover page, certificate, declaration, acknowledgement, abstract, introduction, system analysis, database design, ER diagram, relational schema, table design, normalization, SQL implementation, SQL queries/screenshots, conclusion/future enhancements and references.